STM32F103_串口通信

于 2026-10-15 00:34:34 发布

STM32F103C8T6 串口通信：轮询 / 中断 / DMA 三种方式，附 Keil 逻辑分析仪测波特率

> 这次做的实验是 STM32 串口（USART）。要求挺全：先用 HAL 库实现最基本的串口收发，再把**轮询、中断、DMA** 三种方式都过一遍，最后在没有示波器的情况下用 Keil 的逻辑分析仪看波形、反推波特率。中间踩了不少坑，记录一下留个档。

一、实验是啥

板子还是最常见的 STM32F103C8T6（Blue Pill），内容分"基础 + 三个任务"：

- **基础 1**：串口协议、RS232/RS485/TTL/CH340 这些理论。
- **基础 2**：两台笔记本怎么通过串口传文件。
- **任务一（轮询）**：STM32 往上位机连续发 `hello windows!`，观察"不加延时连续发"时上位机是不是会丢数据；上位机发 `#` 暂停、发 `*` 继续。
- **任务二**：用**中断**方式重做上面的 `#` / `*` 控制，再用 **DMA** 方式连续发送。
- **任务三**：没示波器，用 **Keil 软件仿真 + 逻辑分析仪**看串口输出波形，算实际波特率。

二、先理清串口那点事

帧格式（8N1）

串口是异步通信，双方不共时钟线，靠约定同样的波特率来"对齐"每一位。每个字符 10 个 bit：

```
[起始位 0] [8 个数据位 D0~D7] [停止位 1]
```

AI代码解释

起始位固定是低电平（0），停止位固定是高电平（1），数据位先发低位。波特率 115200 就是每秒 115200 bit，所以 **1 bit ≈ 8.68μs**，一个字符 ≈ 86.8μs。

三种电平：TTL / RS232 / RS485

协议（帧格式、波特率）是一样的，区别在物理层电平：

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

| 标准 | 电平 逻辑 距离 用在哪儿
| TTL | 0~3.3V 高=1 低=0 很短 单片机、CH340 之间
| RS232 | ±15V 负逻辑 ~15m 老式电脑 DB9 串口
| RS485 | 差分 A/B 压差正负 上千米 工业现场、多机总线

单片机串口是 TTL 电平，电脑上只有 USB，所以要用 **CH340** 这个"USB 转 TTL"的桥接芯片：电脑 USB → CH340 → TTL → STM32。CH340 插电脑上被识别成一个 COM 口（我这是 COM3）。

三、两台笔记本怎么传文件

下面讲原理，硬件限制单独标出来。

串口传文件是怎么回事

串口一次只搬一个字节，本身没有"文件"这个概念。"串口传文件"就是把文件当成一串字节，按协议切成一块块发过去，接收方校验对了再拼回文件。麻烦点有两个：

1. **串口没时钟、还可能丢字节**——两边只约定了波特率，数据一多一快就可能错位、丢字节；
2. **怎么知道传完没、传错没**——光靠起止位不够，得在每一块数据里加**序号、长度、校验和**，收方算一遍对不上就让对方重发。

这俩问题就是文件传输协议（XMODEM / YMODEM / ZMODEM）要解决的。

三个文件传输协议

| 协议 | 一块多大 | 校验 | 特点

| XMODEM | 128 字节 | 累加和 | 最老最简单，每块都要等对方 ACK，慢
| YMODEM | 1024 字节 | 累加和 | XMODEM 升级，能一次传多个文件
| ZMODEM | 动态 | CRC32 | 效率最高，带文件名/大小、支持断点续传，最常用

流程都差不多：发送方把文件切成块 → 加序号和校验发出去 → 接收方收到后校验 → 回 ACK（对，继续下一块）或 NAK（错，重发这一块）→ 全部发完再发"结束"标志。ZMODEM 因为自带文件名和断点续传，实际最常用。

硬件怎么接

老台式机/笔记本背后有 **DB9 串口**（公头），两台对连得用**交叉串口线**（也叫空调制解调器线 / null modem）：2 脚 TX 接对方 3 脚 RX、3 脚 RX 接对方 2 脚 TX，GND 共地。现在的笔记本基本没串口了，就换成**两根 USB 转串口**（比如两个 CH340），一根的 TX 接另一根的 RX、RX 接 TX、GND 共地，效果一样。

内容来源：csdn.net
作者昵称：code已满223
作者主页：https://blog.csdn.net/2503_90536812

软件怎么操作

经典做法是用 **Windows 超级终端（HyperTerminal）**，两边设成一样的波特率/数据位/停止位（比如 115200 8N1），接收方点"接收文件"、发送方点"发送文件"，协议选 **ZMODEM**，之后就自动握手、分块传输、校验、结束。

四、轮询方式（任务一）

接线

USART1 用 PA9（TX）/ PA10（RX），串口是**交叉**接的：

- PA9（STM32 发送）→ CH340 模块 **RXD**
- PA10（STM32 接收）→ CH340 模块 **TXD**
- 两边 GND 共地

核心代码

轮询的精髓就在主循环里那句 `HAL_UART_Receive(&huart1, &ch, 1, 0)` ——最后一个参数 `0` 是超时时间，意思是"别等，看一眼有没有字符，没有立刻返回"。CPU 每转一圈就主动问一次串口：

```c
while (1)
{
    /* 非阻塞轮询：查一眼有没有字符，没有就继续 */
    if (HAL_UART_Receive(&huart1, &ch, 1, 0) == HAL_OK)
    {
        if (ch == '#')      g_paused = 1;   /* 收到 '#' 暂停 */
        else if (ch == '*') g_paused = 0;   /* 收到 '*' 继续 */
    }

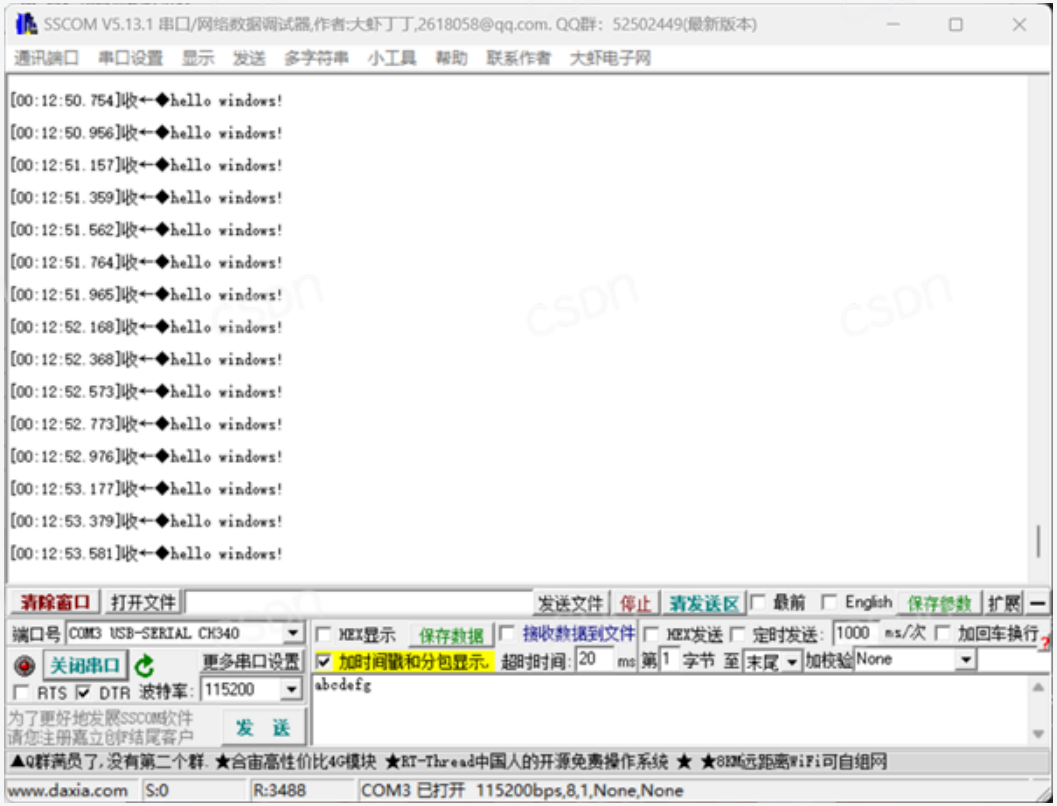
    if (g_paused == 0)
    {
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 1000);
        HAL_Delay(200);                     /* 每句间隔 200ms */
    }
}
```

AI代码解释 C运行

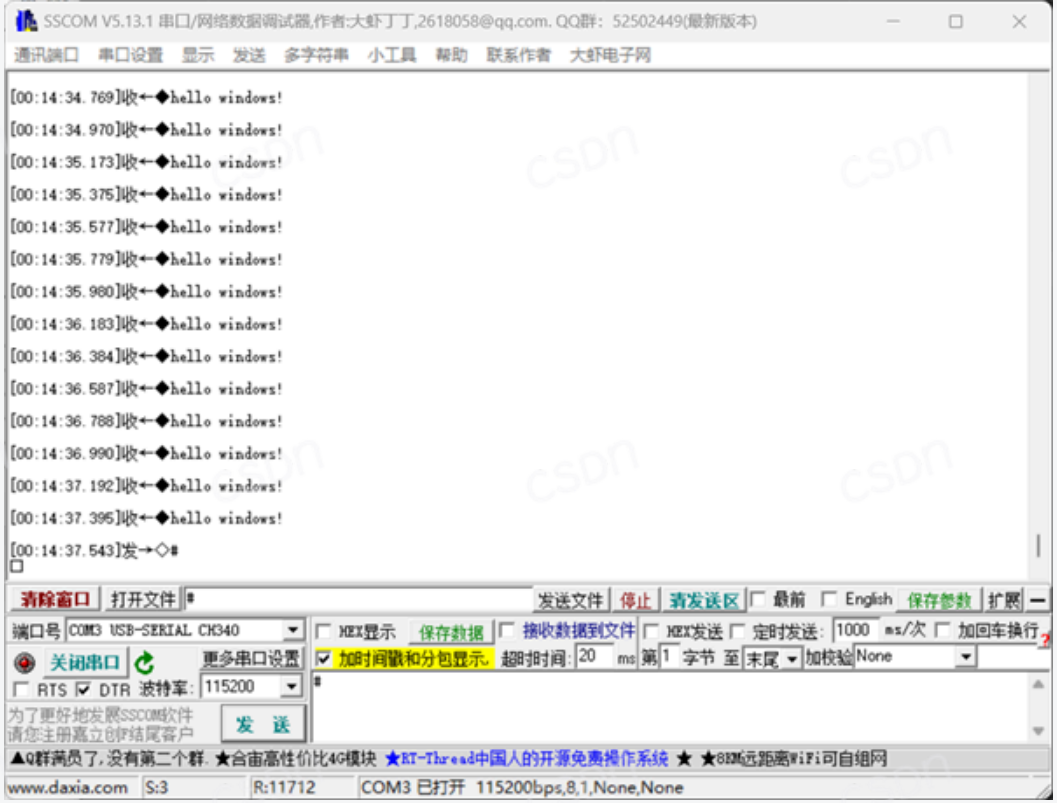
内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

现象

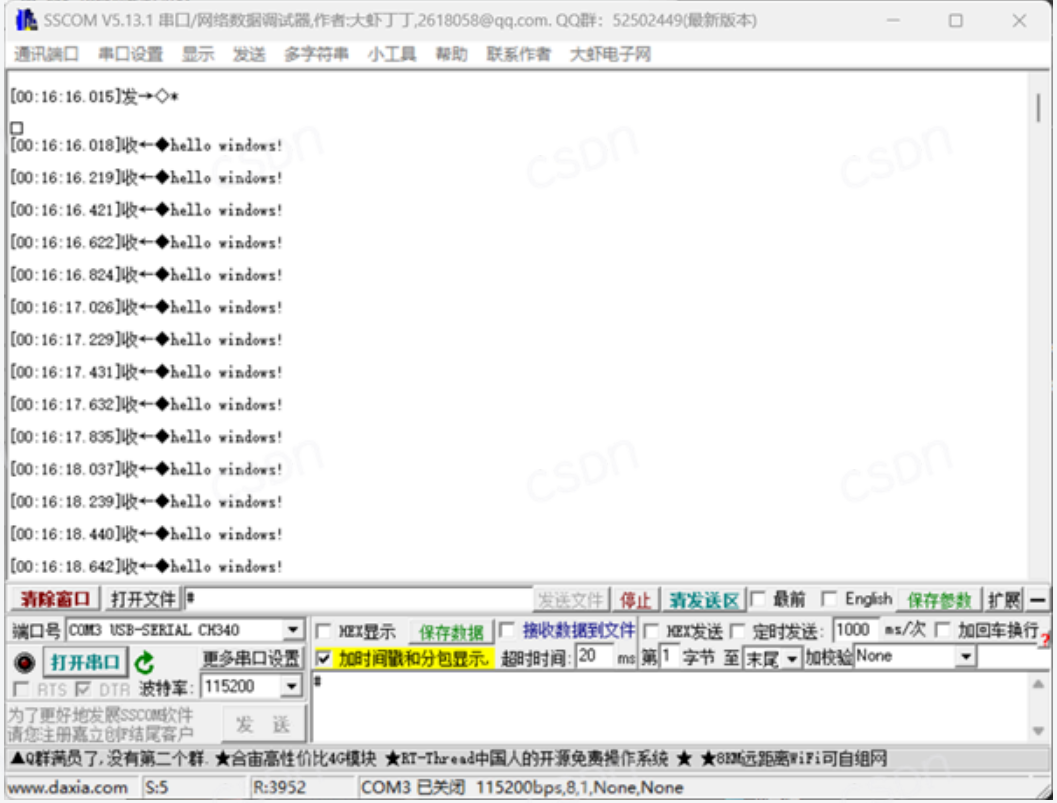
- 加 200ms 延时：串口助手一行行稳定收到 `hello windows!`，不丢不乱；
- 不加延时（把 `HAL_Delay(200)` 去掉）：STM32 最快速度连发，串口助手直接**刷屏**，肉眼可见丢数据——发送快于上位机接收，就会丢。
- `#` 停、`*` 继续，验证通过。



内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812



内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812



五、中断方式

现象跟任务一**一模一样**，但底下机制不同：任务一是主循环主动查字符，任务二是主循环**根本不查**，用 `HAL_UART_Receive_IT` 把接收中断"挂上"，字符一到硬件自动打断CPU、进回调处理。

```c
/* 先启动一次中断接收，之后每收 1 字节都自动进回调 */
HAL_UART_Receive_IT(&huart1, &g_rx_ch, 1);

while (1)
{
    if (g_paused == 0)                    /* 主循环只管发，不查字符 */
    {
        HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 1000);
        HAL_Delay(200);
    }
}
```

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

```c
/* 接收完成回调：HAL 每收满 1 个字节调用一次 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (g_rx_ch == '#')      g_paused = 1;
        else if (g_rx_ch == '*') g_paused = 0;
        HAL_UART_Receive_IT(&huart1, &g_rx_ch, 1);  /* 必须重新开，否则只收一次 */
    }
}
```

AI代码解释 运行

别忘了在 `stm32f1xx_it.c` 里加串口中断入口，和任务一唯一的区别就是 MspInit 里多开了 USART1 的 NVIC 中断：

```c
void USART1_IRQHandler(void)
{
    HAL_UART_IRQHandler(&huart1);
}
```

AI代码解释 运行

六、DMA 方式—— 含"只发一句就停"的坑

DMA 的思路： `HAL_UART_Transmit_DMA` 启动一次，数据由 DMA 硬件直接搬到串口，发完在"发送完成"回调里立刻启动下一句，实现**无延时连续发送**，主循环彻底空着。

USART1 的发送 DMA 固定用 **DMA1 通道 4**。核心代码：

```c
/* 发送完成回调：发完这一句立刻启动下一句（连续发送） */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        g_tx_count++;
        int len = sprintf((char *)tx_buf, "hello windows! DMA send #%lu\r\n",
                          (unsigned long)g_tx_count);
```

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

```
        HAL_UART_Transmit_DMA(&huart1, tx_buf, len);![在这里插入图片描述](https://i-blog.csdnimg.cn/direct/b238d6d2518543f4aebe926a81fe22ee.gif#pic_

    }
}

int main(void)
{
    HAL_Init(); SystemClock_Config(); MX_USART1_UART_Init();
    g_tx_count = 1;
    int len = sprintf((char *)tx_buf, "hello windows! DMA send #%lu\r\n",
                      (unsigned long)g_tx_count);
    HAL_UART_Transmit_DMA(&huart1, tx_buf, len);   /* 启动第一次 DMA */

    while (1) { /* 主循环啥也不用干 */ }
}
```

AI代码解释 运行

坑：只发出 #1 就停了

烧进去之后，串口只蹦出一句 `#1` 就不动了，按一下复位再蹦一句 `#1`。我以为是 DMA 没配好，折腾半天。最后是**翻 HAL 源码**找到的根因：

F1 的 HAL 里，`HAL_UART_Transmit_DMA` 用的是 **DMA 普通模式**，DMA 把最后一个字节搬完后**并不会直接回调** `HAL_UART_TxCpltCallback`，而是干这么一件事——在 `UART_DMATransmitCplt` 里**打开 USART1 的"发送完成 TC"中断**：

```
// stm32f1xx_hal_uart.c 里的 UART_DMATransmitCplt（DMA 普通模式分支）
ATOMIC_CLEAR_BIT(huart->Instance->CR3, USART_CR3_DMAT);    // 关 DMA 请求
ATOMIC_SET_BIT(huart->Instance->CR1, USART_CR1_TCIE);      // 开 UART 的 TC 中断
```

AI代码解释 运行

然后等 UART 把最后一个字节真正移位发完，才由 **USART1 中断**收尾：`USART1_IRQHandler → HAL_UART_IRQHandler → UART_EndTransmit_IT → 回调 HAL_UART_TxCpltCallback`。

我一开始只写了 `DMA1_Channel4_IRQHandler`，**没写** `USART1_IRQHandler`，于是 DMA 搬完 → 打开 TC 中断 → 结果 USART1 中断没人接，程序直接卡死，所以只有第一句。补上这两处就好了：

内容来源：csdn.net
作者昵称：阿司匹林229
作者主页：https://blog.csdn.net/2503_90536812

```c
/* MspInit 里，除了 DMA 中断，还要开 USART1 中断 */
HAL_NVIC_SetPriority(DMA1_Channel4_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(DMA1_Channel4_IRQn);
HAL_NVIC_SetPriority(USART1_IRQn, 0, 0);   // 关键！不能少
HAL_NVIC_EnableIRQ(USART1_IRQn);
```

AI代码解释 c运行

```c
/* stm32f1xx_it.c 里两个中断都得有 */
void DMA1_Channel4_IRQHandler(void) { HAL_DMA_IRQHandler(&hdma_usart1_tx); }
void USART1_IRQHandler(void)        { HAL_UART_IRQHandler(&huart1); }
```

AI代码解释 c运行

补完就能看到计数 `#1 #2 #3 …` 一路狂飙了。

体会：HAL 库是"封装"不是"黑盒"，遇到异常就得敢去翻源码，把中断是怎么链起来的搞清楚，不然光靠猜能卡一晚上。



七、三种方式到底差在哪

三个版本波特率都是 115200（每个 bit 一样快），区别在"每句之间隔多久"以及"谁在搬数据"：

```
                轮询                中断                DMA
```

AI代码解释

| 接收 主循环主动查 硬件打断 （本实验只演示发送）
| 发送 CPU 阻塞发 CPU 阻塞发 **DMA 硬件搬**
| 主循环 边发边查 只管发 **空的**

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

| 每句间隔 200ms 200ms **无延时，连发**
| 输出速度 ~5 句/秒 ~5 句/秒 **几千句/秒**

一句话：**轮询是 CPU 追着数据跑，中断是数据到了把 CPU 拽过来，DMA 是数据自己搬、CPU 撒手不管。** 所以 DMA 版没有 200ms 延时、由硬件+中断连续接力，输出快了几百倍——这也是为什么老师问"DMA 输出为什么那么快"。

八、Keil 逻辑分析仪测波特率

配置步骤

没示波器也能看时序，Keil 自带软件仿真：

1. `Options for Target → Debug` 左边选 **Use Simulator**；
2. 仿真器的 Dialog DLL 填 `DARMSTM.DLL`、参数 `-pSTM32F103C8`（**不填这个仿真器不会加载 STM32 外设模型，逻辑分析仪认不到引脚**）；
3. F7 编译，`Ctrl+F5` 进调试（纯仿真，不连 ST-Link），`F5` 运行；
4. `View → Analysis Windows → Logic Analyzer`；
5. Setup 里加信号，Signal 填 `PORTA.9`（PA9 = USART1_TX），Display Type 选 `Bit`；
6. Run 看波形。

坑：逻辑分析仪报 Unknown Signal

两个原因我都踩了：

- **信号名写错**：一开始填 `GPIOA->ODR`，逻辑分析仪直接报 `Unknown Signal`。Keil 的逻辑分析仪不认这种指针写法，要用它自己的端口命名 `PORTA.9`（`PORTA.x` 这种，跟 8051 的老命名一样）。
- **DLL 没配**：切到 Use Simulator 后，仿真器的 Dialog DLL 是空的，得手动补 `DARMSTM.DLL -pSTM32F103C8`。

算波特率

PA9 空闲是高电平，发一个字符就是"拉低 1 bit（起始）→ 8 个数据位 → 拉高 1 bit（停止）"。量最窄的一段电平宽度就是 1 个 bit，实测约 **8.68μs**。

理论也能精确算出来。USART1 在 APB2 上，PCLK2 = 72MHz：

```
波特率 = PCLK2 / (16 × USARTDIV)
USARTDIV = 72000000 / (16 × 115200) = 39.0625
```

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812

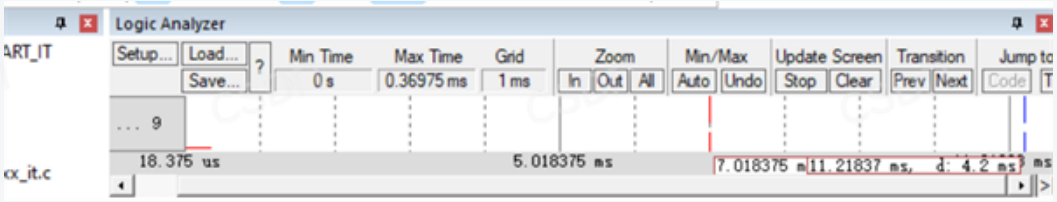
AI代码解释

`39.0625` 的整数部分 39 放进 BRR 的 12 位整数，小数 0.0625 = 1/16 正好放进 4 位小数位，所以 `BRR = 0x271`，实际波特率：

```
72000000 / (16 × 39.0625) = 115200    （精确，误差 0%）
```

AI代码解释

原因就是 72MHz/16 = 4.5MHz，被 39.0625 整除，而且 0.0625 正好是 1/16，4 位小数位能精确表示。实测 8.68μs 反推也是 115200，跟理论完全对上。



九、踩过的坑（汇总）

- **串口完全没反应**：先查 CH340 模块插没插。设备管理器里 COM 口显示"幽灵设备"（CM_PROB_PHANTOM）就是模块没插到电脑。
- **烧录后不运行**：Flash Download 里没勾 **Reset and Run**，烧完芯片停在复位态。勾上，或烧完按一下板子 RESET。
- **DMA 只发一句就停**：缺 `USART1_IRQHandler`（详见第六节，这个最隐蔽）。
- **逻辑分析仪 Unknown Signal**：信号名用 `PORTA.9` 别用 `GPIOA->ODR`，且 Dialog DLL 要配 `DARMSTM.DLL -pSTM32F103C8`。
- **Keil AC6 中文路径报错**：用户名是中文的话，armclang 编译默认启动文件会报 “Unable to convert command-line to UTF-16”，用 GNU 语法启动文件 + MicroLIB 能绕开。

附：AI 在实验里帮了什么忙

这次实验我用了大模型 AI（Claude）辅助，主要帮了这些：

1. **讲原理**：串口帧格式、TTL/RS232/RS485 区别、轮询/中断/DMA 三者区别，AI 用类比（“轮询像反复开门看快递，中断像快递按门铃”）讲得挺清楚。
2. **搭代码框架 + 注释**：三个工程的 HAL 代码骨架和逐段中文注释，让我能看懂每行在干嘛。
3. **排错（帮最大）**：几个坑都是它帮我定位的——串口没反应查出是 CH340 没插、烧录不运行是没勾 Reset and Run、DMA 只发一句是缺 USART1 中断（带我一起翻 HAL 源码）、逻辑分析仪 Unknown Signal 是信号名和 DLL 的问题。
4. **算波特率 + 整理文档**：72MHz 下 115200 的 USARTDIV=39.0625、BRR=0x271 的完整推导，以及这份博客的结构。

作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
版权：https://blog.csdn.net/2503_90536812

反思：AI 的结论我没有全盘照抄，DMA 那个坑是跟着它一起**读 HAL 源码**、自己看懂中断链路才改对的。AI 能帮你更快定位问题、查资料、整理文档，但真正掌握原理、判断它说得对不对，还得靠自己去读代码、动手验证。合理用 AI 能省不少时间，但不能替代自己动手和思考。

标签
#stm32 #单片机 #嵌入式硬件

内容来源：csdn.net
作者昵称：阿司匹林229
原文链接：https://blog.csdn.net/2503_90536812/article/details/167920771
作者主页：https://blog.csdn.net/2503_90536812